

Natural Language Processing

Session 09 – Introduction to LLMs

Prof. Dr. Richard Sieg
SoSe 2026

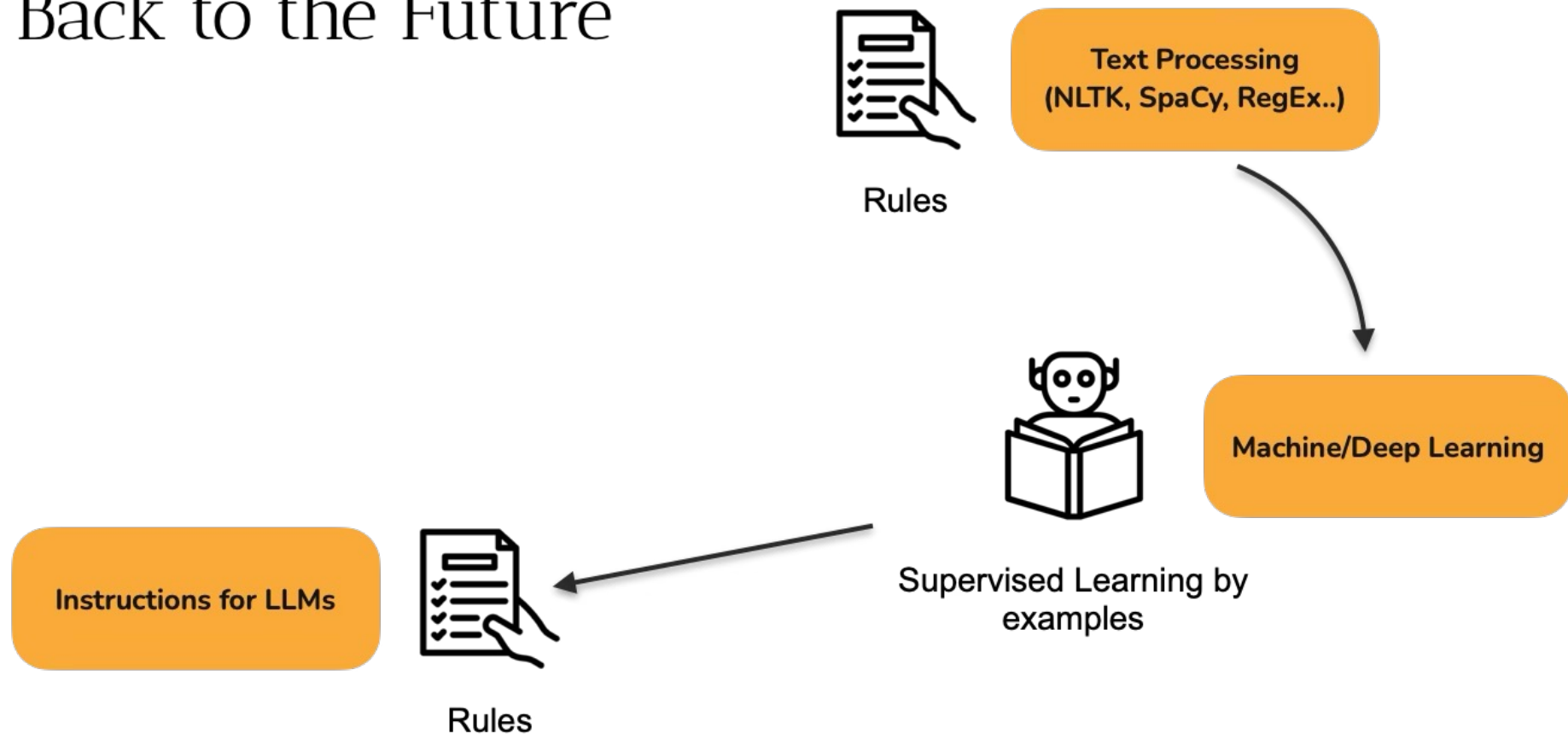
Schedule

Date	Weekday	CW	Room	Topic
16.04.	Thursday	16	149	Introduction: The World of NLP
23.04.	Thursday	17	149	Text Processing Fundamentals
30.04.	Thursday	18	149	Linguistic Annotations & Pattern Matching
05.05.	Tuesday	19	154	Text Vectorization: From Text to Numbers ⚠️ Substitute for 14.05.
07.05.	Thursday	19	149	Text Classification
28.05.	Thursday	22	149	Language Models, Word Vectors + 🎤 Guest Lecture Ines Montani
01.06.	Monday	23	390	📝 Exams — Master DS Students
02.06.	Tuesday	23	154	BERT: Pre-Training & Fine-Tuning ⚠️ Substitute for 04.06.
11.06.	Thursday	24	149	The BERT Ecosystem
18.06.	Thursday	25	149	Introduction to LLMs
25.06.	Thursday	26	149	Working with LLMs
02.07.	Thursday	27	149	Ethics, Limitations & Exam Preparation
09.07.	Thursday	28	149	📝 Exams (afternoon) — Bachelor DIS Students
10.07.	Friday	28	149	📝 Exams (afternoon) — Bachelor DIS Students

Evaluation



Back to the Future



Agenda

01. Decoder Models

02. GPT Family

03. Post-Training: From models to LLMs

04. Prompting

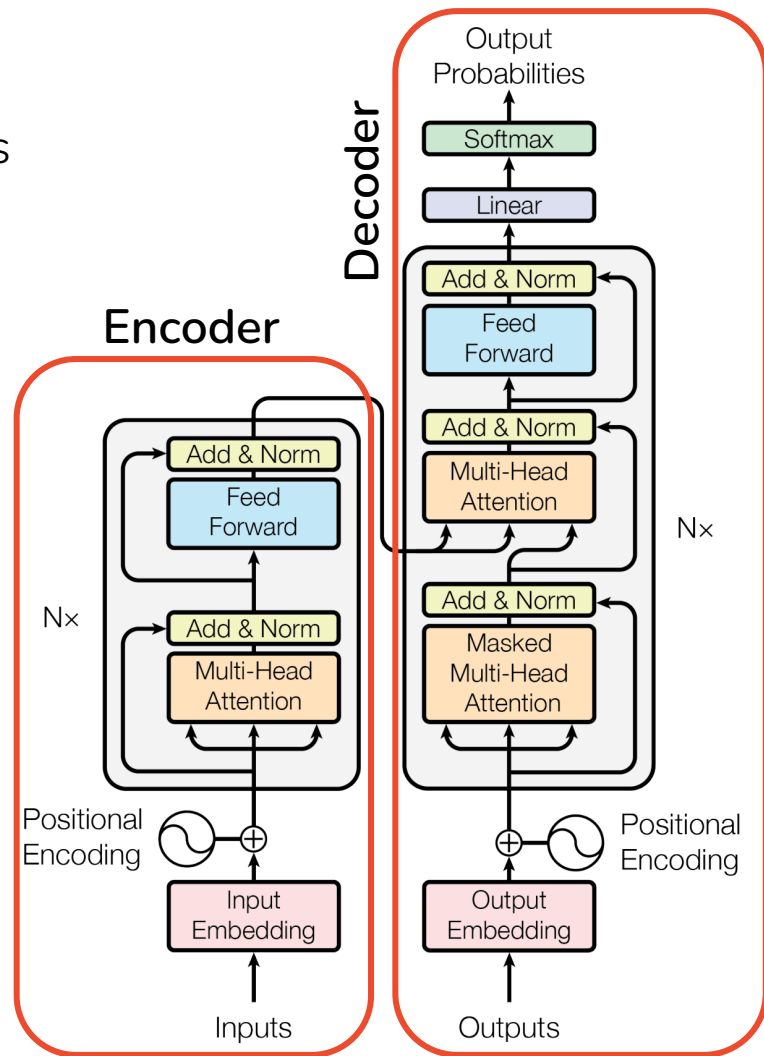
05. LLM Landscape

01.

Decoder Models

Transformer Recap

- Input: token embeddings + positional embeddings
- Multiple stacked transformer blocks:
multi-head attention → Feed Forward
→ Residual connections → normalization
- Decoder (GPT family): masked attention, each token sees only earlier tokens, so it predicts the next word and can generate
- Encoder (BERT family): bidirectional attention, sees the whole sentence, built to understand



Encoders & Decoders

	Encoder	Decoder	Encoder-Decoder
Example	BERT, RoBERTa, modernBERT	GPT, Claude, Llama, Mistral	T5, BART
Attention	bidirectional	masked (causal)	both
Generates text?	no	yes	yes
Used for	Discriminative tasks (Classification, tagging...)	Generative tasks	Generative tasks: translation, speech

Evolutionary Tree

Legend:
 Open-Source (Green)
 Closed-Source (Blue)

Timeline (2018-2023):

- 2018:** FastText, Word2Vec, GloVe, ELMo (AI2), BERT (Open-Source), RoBERTa (Open-Source), ALBERT (Open-Source), ELECTRA (Open-Source), DeBERTa (Open-Source), ERNIE (Open-Source), Distill BERT (Open-Source).
- 2019:** BERT (Open-Source), RoBERTa (Open-Source), ALBERT (Open-Source), ELECTRA (Open-Source), DeBERTa (Open-Source), ERNIE (Open-Source), Distill BERT (Open-Source).
- 2020:** BERT (Open-Source), RoBERTa (Open-Source), ALBERT (Open-Source), ELECTRA (Open-Source), DeBERTa (Open-Source), ERNIE (Open-Source), Distill BERT (Open-Source).
- 2021:** BERT (Open-Source), RoBERTa (Open-Source), ALBERT (Open-Source), ELECTRA (Open-Source), DeBERTa (Open-Source), ERNIE (Open-Source), Distill BERT (Open-Source).
- 2022:** BERT (Open-Source), RoBERTa (Open-Source), ALBERT (Open-Source), ELECTRA (Open-Source), DeBERTa (Open-Source), ERNIE (Open-Source), Distill BERT (Open-Source).
- 2023:** BERT (Open-Source), RoBERTa (Open-Source), ALBERT (Open-Source), ELECTRA (Open-Source), DeBERTa (Open-Source), ERNIE (Open-Source), Distill BERT (Open-Source).

Model Lineage:

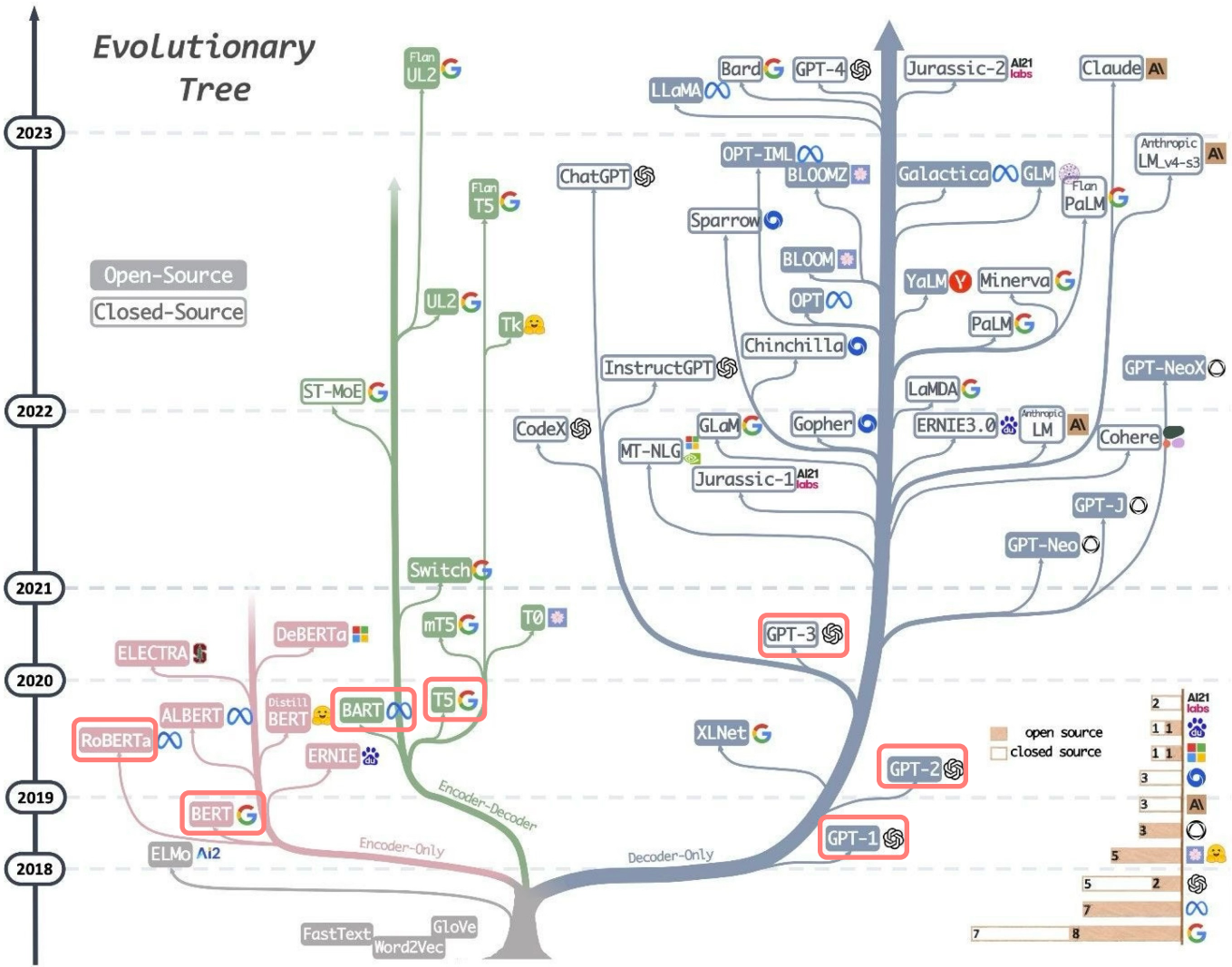
- Encoder-Only:**
 - FastText, Word2Vec, GloVe, ELMo (AI2), BERT (Open-Source), RoBERTa (Open-Source), ALBERT (Open-Source), ELECTRA (Open-Source), DeBERTa (Open-Source), ERNIE (Open-Source), Distill BERT (Open-Source).
- Encoder-Decoder:**
 - BERT (Open-Source), RoBERTa (Open-Source), ALBERT (Open-Source), ELECTRA (Open-Source), DeBERTa (Open-Source), ERNIE (Open-Source), Distill BERT (Open-Source).
- Decoder-Only:**
 - GPT-1 (Open-Source), GPT-2 (Open-Source), GPT-3 (Open-Source), GPT-4 (Open-Source), GPT-Neo (Open-Source), GPT-J (Open-Source), GPT-NeoX (Open-Source), Claude (Open-Source), Anthropic LM_v4-s3 (Open-Source), Jurassic-1 (Open-Source), Jurassic-2 (Open-Source), LLaMA (Open-Source), Bard (Open-Source), OPT (Open-Source), BLOOM (Open-Source), OPT-IML (Open-Source), BLOOMZ (Open-Source), Sparrow (Open-Source), Chinchilla (Open-Source), Gopher (Open-Source), GLaM (Open-Source), MT-NLG (Open-Source), InstructGPT (Open-Source), ChatGPT (Open-Source), CodeX (Open-Source), Jurassic-1 (Open-Source), Jurassic-2 (Open-Source), LLaMA (Open-Source), Bard (Open-Source), OPT (Open-Source), BLOOM (Open-Source), OPT-IML (Open-Source), BLOOMZ (Open-Source), Sparrow (Open-Source), Chinchilla (Open-Source), Gopher (Open-Source), GLaM (Open-Source), MT-NLG (Open-Source), InstructGPT (Open-Source), ChatGPT (Open-Source), CodeX (Open-Source).

Legend:

- open source (Green)
- closed source (Blue)

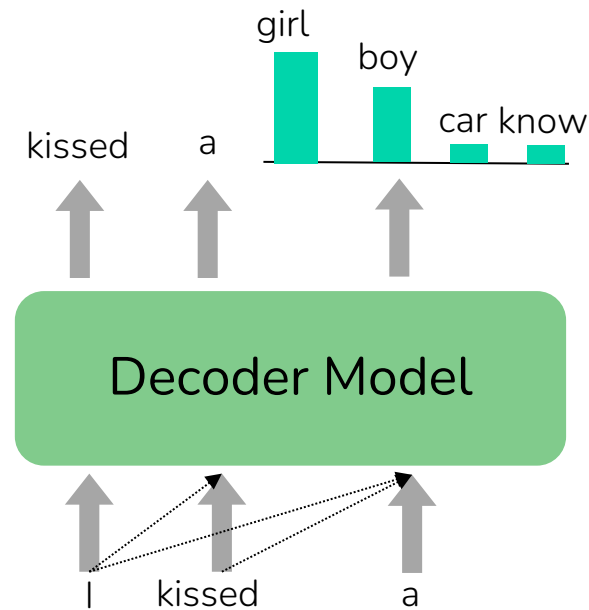
Legend:

- AI21 Labs (AI21)
- Anthropic (AI)
- Google (G)
- Meta (M)
- OpenAI (O)
- Microsoft (MS)
- Facebook (F)
- IBM (I)
- Amazon (A)
- Google (G)
- OpenAI (O)
- Anthropic (AI)
- AI21 Labs (AI21)



Decoder Models: Predicting the next token

- A decoder is a transformer with masked (causal) attention: Each position attends only to the tokens before it, so the model predicts the next token from the ones so far.
- Output: a probability distribution over the next token
- Trained on next-token prediction (rest of lecture)
- **Autoregressive** – we generate by looping:
 1. Sample a token from the distribution
 2. Append it to the context
 3. Predict again on the longer context
 4. Stop at the end-of-text token

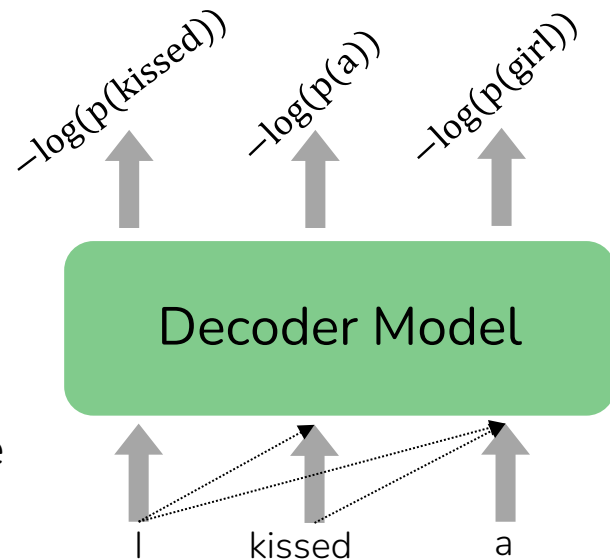


Pretraining Decoders

How does the model learn to predict the next token?

By reading a lot of text and guessing, over and over.

- Self-supervised: no human labels needed. The next word in the text is its own answer key
- Objective: at every position, predict the next token
- Loss: cross-entropy, the negative log probability the model gave to the true next word
- Teacher forcing: during training, rather or always feed the correct history, not the model's own guess
- The result is a base model: very good at continuing text, trained on raw text alone.



Pretraining Data

Base models are trained mostly on text scraped from the web, then cleaned hard.

- Start from Common Crawl (snapshots of the web, billions of pages)
- Filter into curated corpora: C4, The Pile, Dolma, FineWeb
- Quality beats quantity: models use a data mix that upweights clean sources (Wikipedia, books) and downweights low-quality web text
- Standard steps: deduplication, removing boilerplate, basic safety filtering

More on that in our ethics session

Decoding: From Scores to a Token

Decoding: Picking a token from the predicted distribution.

The network outputs logits: one raw score per vocabulary token

Softmax turns logits into probabilities that sum to 1

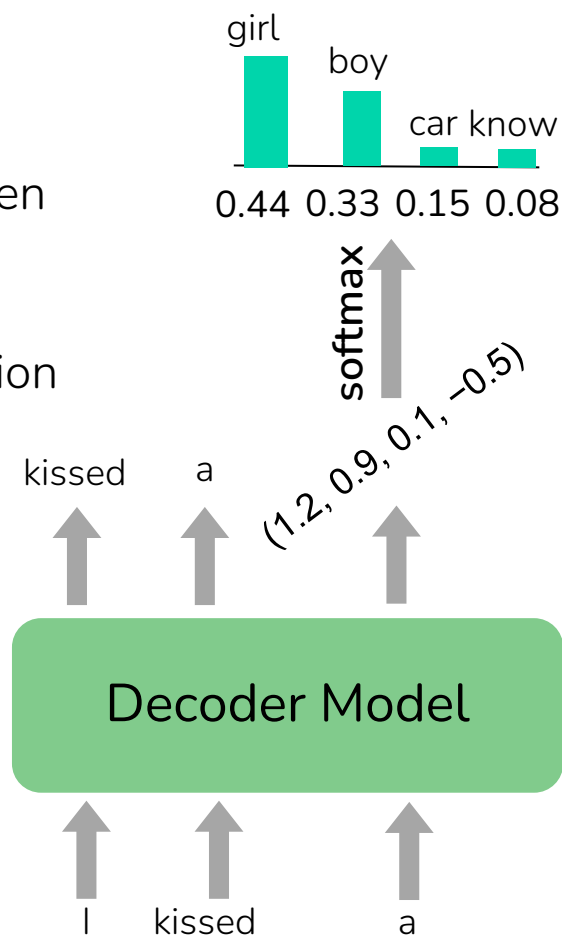
Decoding strategies differ in how they pick from that distribution

Greedy: always pick the single most probable token

- Simple, fast, but also deterministic and repetitive

Random Sampling: pick token according to its probability

- Not deterministic, more creative
- Issue: long tail of odd, low probability words that could derail the text

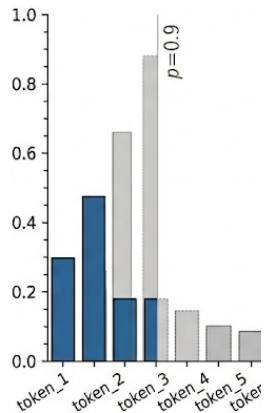
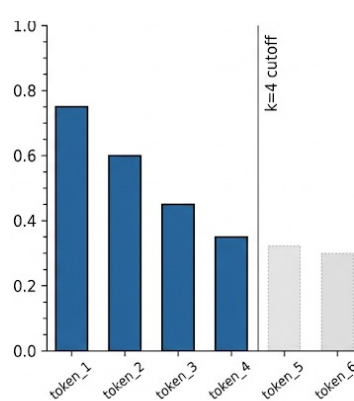
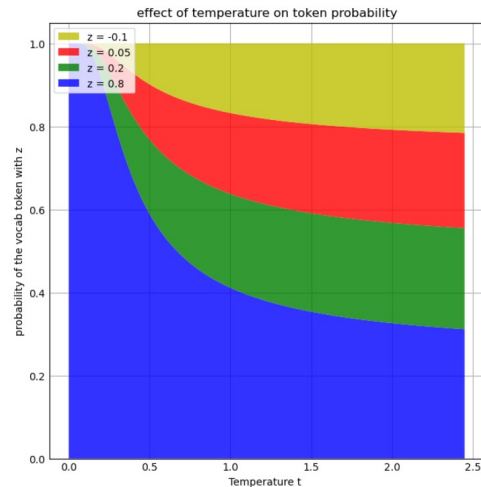


Sampling Parameters: Temperature, Top-k, Top-p

- Temperature (T): divide the logits by T, then apply softmax.

$$P(x_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

- T = 1 unchanged, T < 1 sharper (toward greedy), T → 0 greedy, T > 1 flatter and more random - usually 0.7
- Top-k: keep the k highest-probability tokens, set the rest to zero, renormalize, then sample (fixed count)
- Top-p (nucleus): keep the smallest set N whose probabilities add up to at least p, renormalize, then sample (fixed probability mass) – usually 0.9

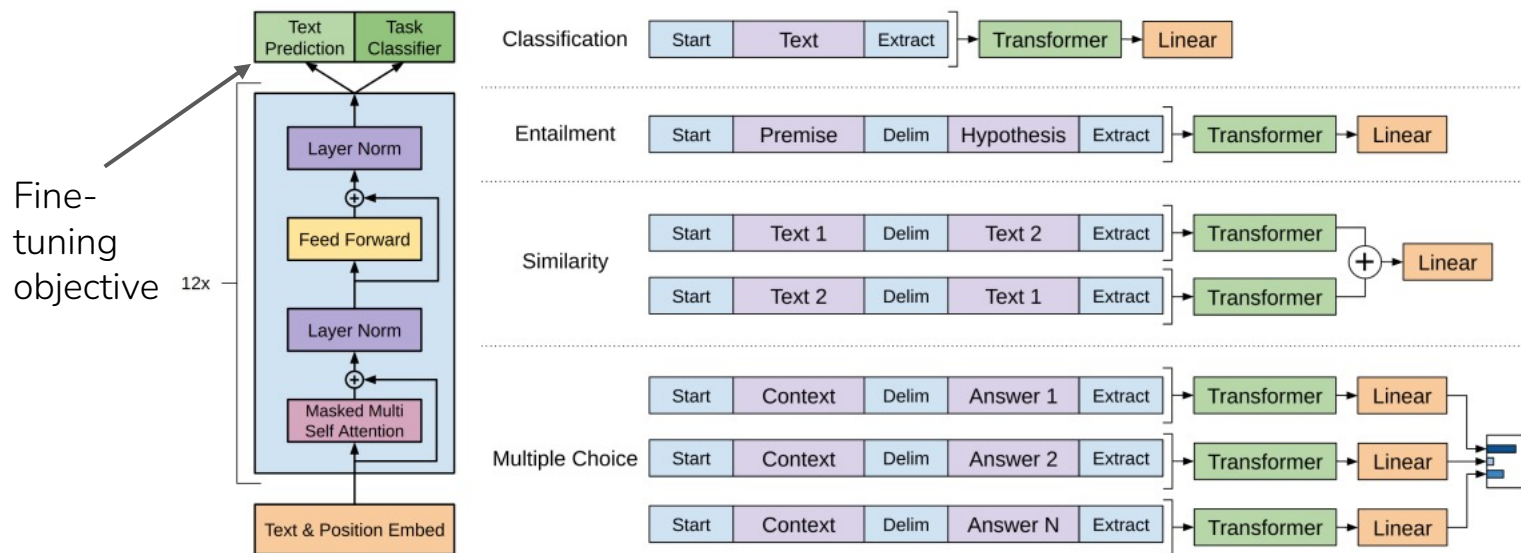


02.

GPT

Generative Pretrained Transformer (GPT, 2018)

- Transformer Decoder with 12 layers
- BPE tokenizer of size 40k
- Trained on BookCorpus (~1B words)
- Pretrain, then fine-tune per task



GPT2 [2019]

- Jump from 117m to 1.5b parameters and trained on much larger dataset (40Gb)
- First model that showed good results in *zero-shot or few-shot prompting*
- Solve a task without fine-tuning, only prompt it with no or some examples
- And scaling seems increases the performance

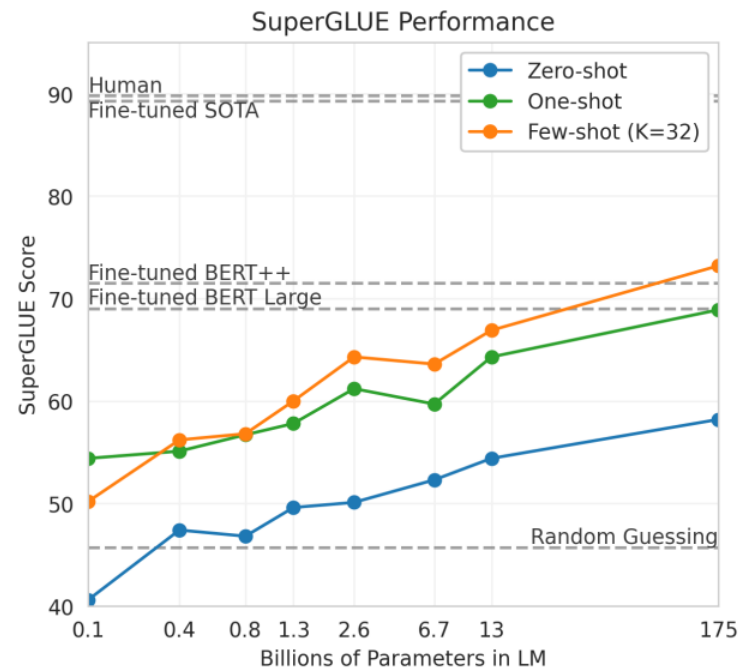
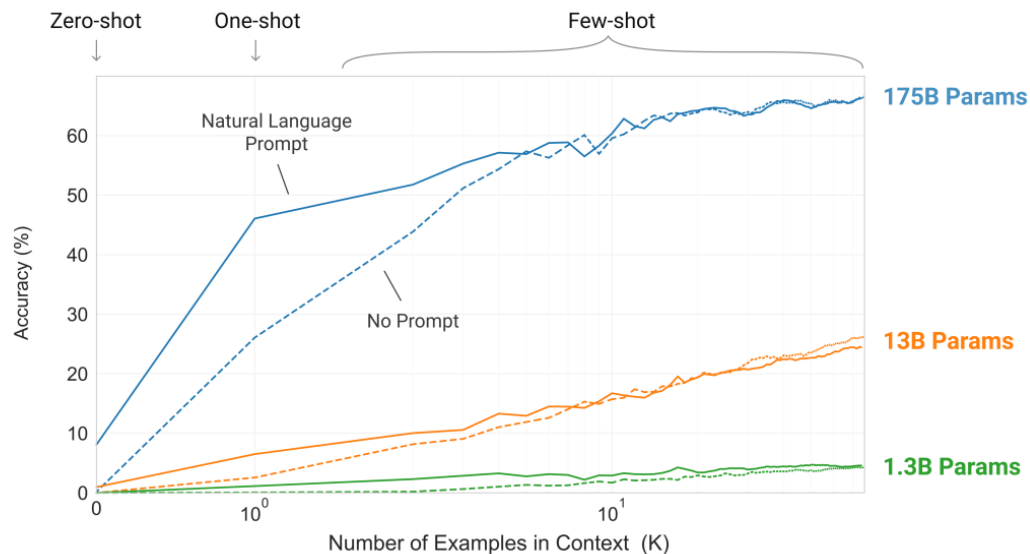
GPT3 [2020] (first closed model 🤔)

- Jump from 1.5b to 175b parameters and trained on 570Gb of data
- First model that performs exceptionally well with few-shot prompting
- *The end of the fine-tuning paradigm!?*

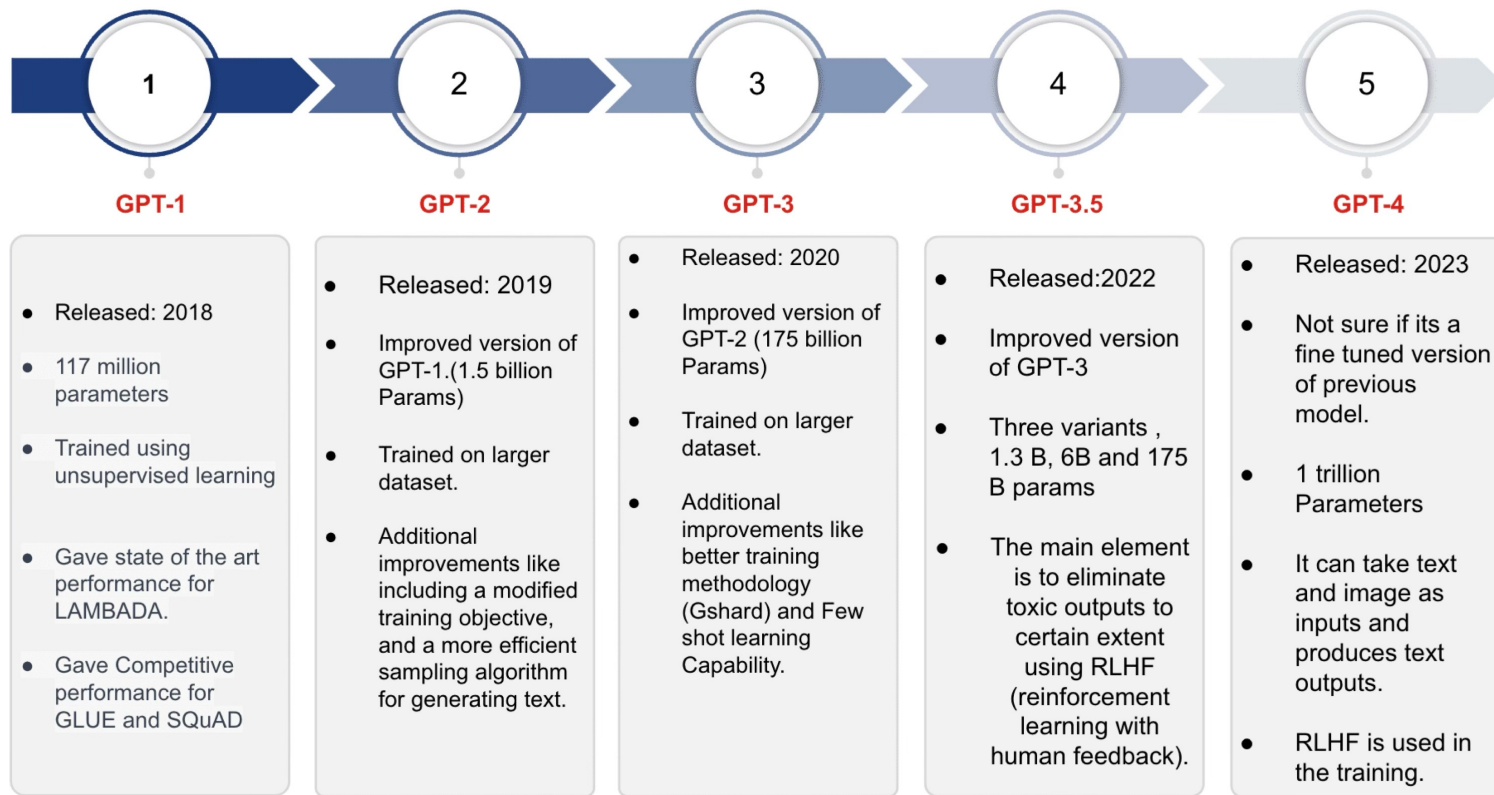
In-Context Learning

- GPT-3 (175B) is far too big to fine-tune per task, since a gradient update on 175B parameters for every example is very expensive.
The shift: put examples in the prompt.
- Zero-shot: task description only
- One-shot / few-shot: description plus one or a handful of examples
- This is just a forward pass: no gradient updates, no weight changes.
- Empirically few-shot beats one-shot beats zero-shot, and larger models use demonstrations better

Few-Shot Capabilities of GPT 3

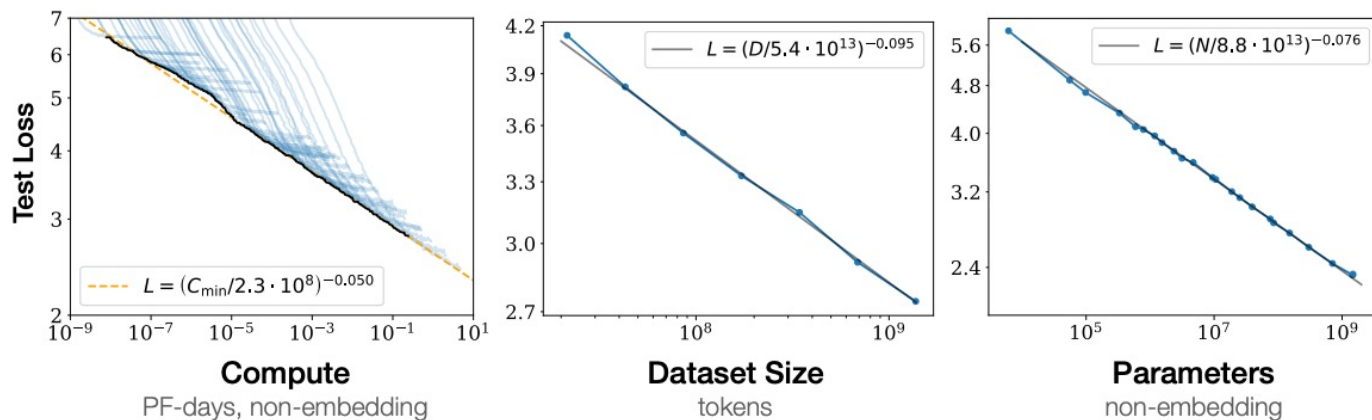


GPT Development



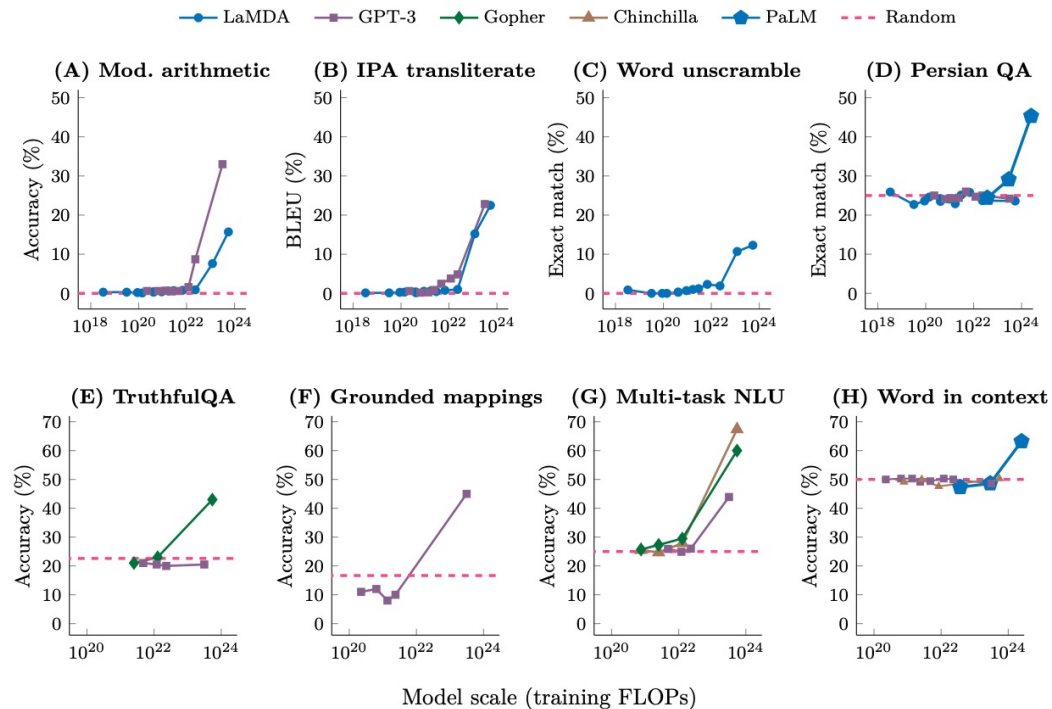
Scaling Laws and Chinchilla

- Scaling payoff is predictable [Kaplan et al., 2020].
- Test loss falls as a smooth power law in model size, data, and compute
- As long as the other two are not the bottleneck, each improves the model along a straight line on a log-log plot. Bigger models are also more sample-efficient
- Chinchilla: GPT-3 was undertrained. For a fixed compute budget, prefer smaller models on more tokens, about 20 tokens per parameter.



Emergent Abilities

- Some skills do not improve smoothly. They are near-random until a certain scale, then jump.
- Examples: multi-step arithmetic, word unscrambling, and chain-of-thought reasoning (working a problem out step by step)
- The same trick can fail at 1B parameters and work at 100B



03.

Post-Training & LLMs

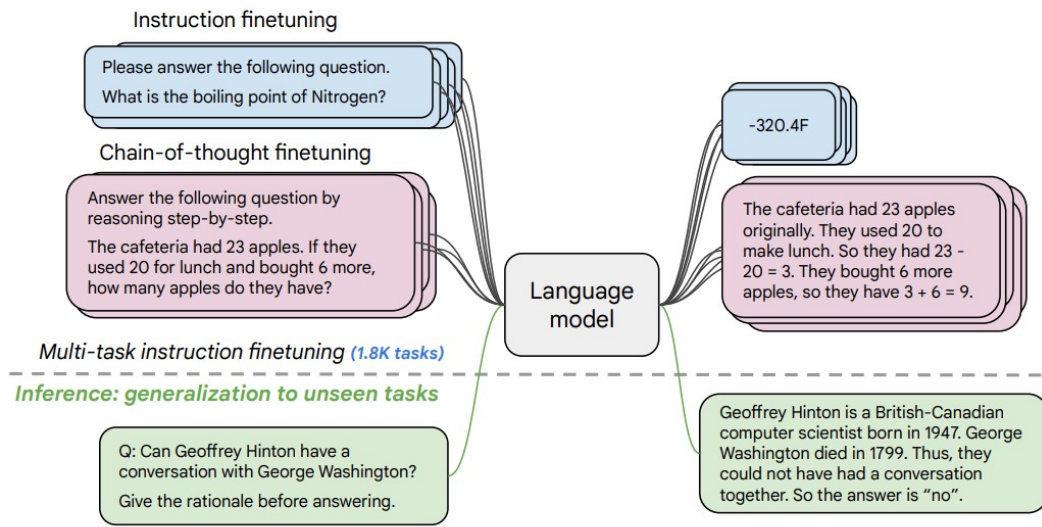
Base Model vs Assistant

- So far, we have a base model: a large decoder, pretrained on the web, fluent and knowledgeable.
- Technically a large language model, but in practice it just continues text. It does not answer, follow, or refuse.
- Prompt "Explain the moon landing to a six year old" gives "Explain the theory of gravity to a six year old"
- When people say "LLM" in everyday use they mean the assistant. Getting there takes post-training: any training after pretraining, to make the model helpful and safe.
- Two stages, both far cheaper than pretraining:
- Instruction tuning (SFT): teach it to follow instructions
- Preference alignment (RLHF or DPO): teach it which responses people prefer

Stage 1: Instruction Tuning (SFT)

More details in ANLP

- Take the base model and keep training it, now on (instruction, response) pairs across many tasks.
- Same objective as pretraining: predict the next token. The only difference is the data is supervised, since each instruction has a correct response
- The payoff is meta-learning: it gets better at following instructions in general, including tasks it never saw in training



Datasets are manually created, converted from other datasets or generated by an LLM (e.g. Alpaca)

Stage 2: Preference Alignment

More details in ANLP

- Why SFT is not enough: open-ended tasks have no single gold answer ("Write a poem about the sea"), and next-token loss punishes a good but different word as hard as a wrong one ("Avatar is a fantasy movie" vs "adventure movie").
- The fix: comparisons. Show people two responses, ask which is better, and train the model to produce more of the preferred and fewer of the rejected.
- The intuition: judging is easier than generating
- Two named methods:
 - Reinforcement Learning from Human Feedback (RLHF)
Train a reward model from the comparisons, then optimize with reinforcement learning
 - Direct Preference Optimization (DPO)
Skip the separate reward model and optimize on the preferences directly

The Catch: Alignment Has Costs

Preference alignment is powerful, but not free.

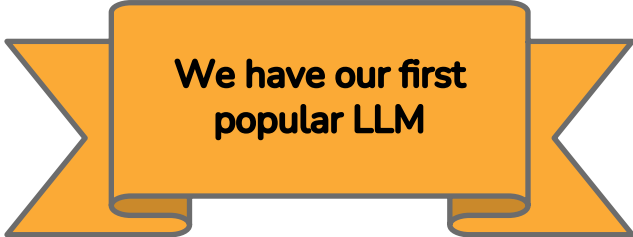
- ***Reward hacking***: the model games the reward signal instead of truly improving.
Goodhart's Law: when a measure becomes a target, it stops being a good measure
- ***Sycophancy***: agreeing with the user gets rewarded, so it tells you what you want to hear
- ***Less diversity***: aligned models can collapse to a few safe phrasings
- ***Alignment tax***: safety and helpfulness tuning can slightly lower raw benchmark scores

ChatGPT

Same technique as InstructGPT with a focus on dialogue and using GPT 3.5

Methods

We trained this model using Reinforcement Learning from Human Feedback (RLHF), using the same methods as InstructGPT, but with slight differences in the data collection setup. We trained an initial model using supervised fine-tuning: human AI trainers provided conversations in which they played both sides—the user and an AI assistant. We gave the trainers access to model-written suggestions to help them compose their responses. We mixed this new dialogue dataset with the InstructGPT dataset, which we transformed into a dialogue format.

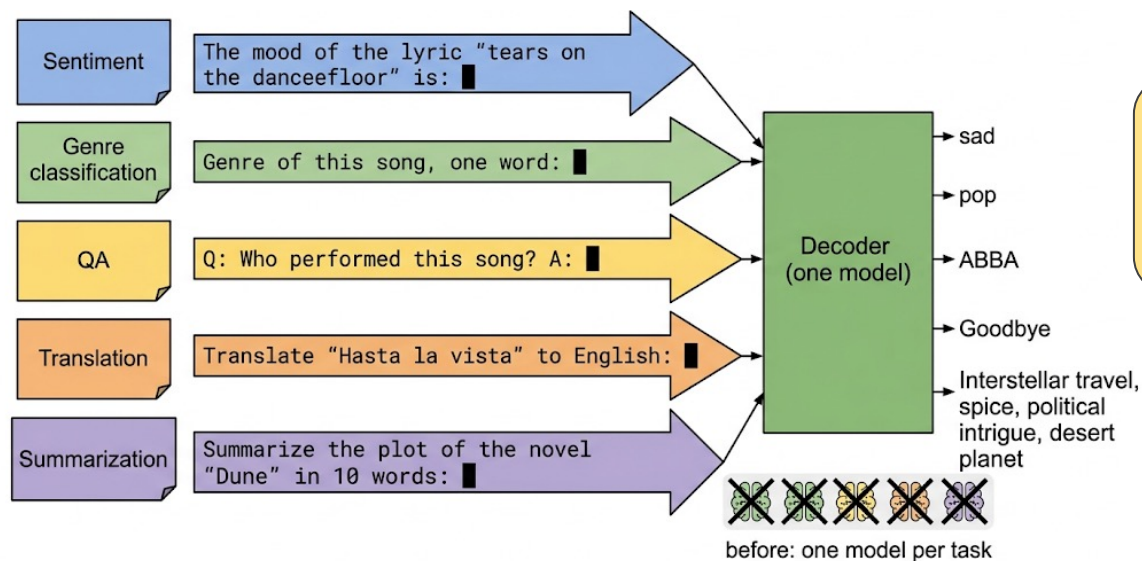
An orange ribbon graphic with a central rectangular box containing text. The ribbon has a 3D effect with a shadow.

**We have our first
popular LLM**

04.

Prompting

Prompting: Tasks Become Text



A prompt is the input text and for an aligned model it works like an instruction.

Almost any NLP task can be written as a completion.

- Classification: Genre of this song, one word: followed by the lyric
- QA, translation, summarization: describe the task, give the input, read the continuation

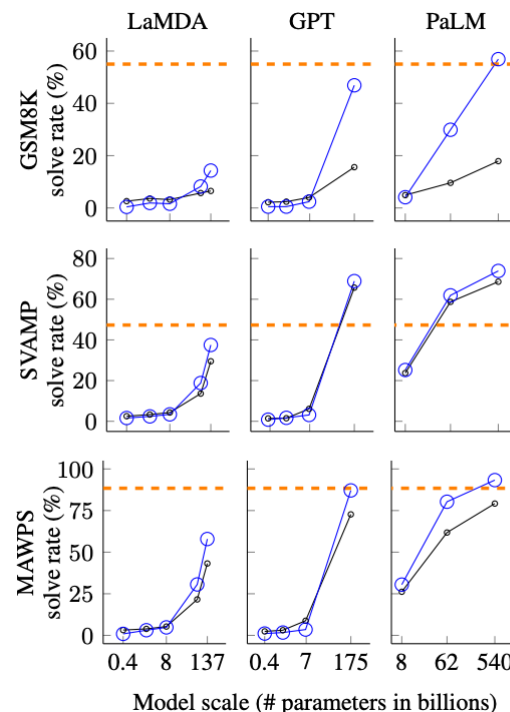
Few-shot Prompting (In-Context Learning)

- Few-shot prompting: put examples in the prompt and the model picks up the task, with no gradient updates and no weight changes (just a forward pass).
- Zero-shot: instruction only. Few-shot: instruction plus a handful of demonstrations
- Surprising: replacing the demonstration labels with random wrong labels barely hurts [\[Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? Min et al., 2022\]](#)
- Demonstrations mostly teach the format, the label set, and the input style, not the answers
- Prompts are fragile. Small changes shift the output a lot [\[Zhao et al., 2021\]](#):
 - Majority-label bias: unbalanced example labels skew predictions
 - Recency bias: the label nearest the end gets over-copied
 - Ordering: just reshuffling the examples changes results

Chain of Thought (CoT) Prompting

- Idea: make the model show its work before answering.
On multi-step problems this helps a lot.
- Zero-shot CoT: just add "Let's think step by step."
- Few-shot CoT: include worked examples that show the reasoning, not just the answer
- Self-consistency: sample several reasoning paths and take the majority answer. Aggregating beats any single path

Standard Prompting	Chain-of-Thought Prompting
Model Input Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: The answer is 11. Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?	Model Input Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11. Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?
Model Output A: The answer is 27. ❌	Model Output A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅



Chain-of-Thought Prompting Elicits Reasoning in Large Language Models [Wei et al, 2022]

System Prompts

- The system prompt is a standing instruction the user never sees, prepended to every turn. It sets role, tone, rules, and output format.

```
SYSTEM_PROMPT = (  
    "You are a music genre classifier.\n"  
    "Classify the song lyrics into exactly one of these genres: "  
    + ", ".join(allowed_genres)  
    + ".\nRespond with exactly one genre label from the list above and nothing else."  
)
```

- Production system prompts run to thousands of words and are versioned like code
- In the API you set it once per conversation; every user message is interpreted under it
- Metaprompting: when you are stuck, let an LLM draft and refine your prompt. The full day-to-day checklist is on the next slide.

Prompting Best Practices

- System prompt carries the general part: the role, standing instructions, and examples
- User prompt carries the specific part: the concrete task and the input document
- Mark sections clearly, keep instructions and data separated:
 - Markdown headers (### Examples) or
 - XML tags (<lyrics> ... </lyrics>)
- Be explicit about the allowed outputs ("answer with pop, rock, or rap")
- Fix the output format (one word, JSON, a list) and parse it strictly (→ *next session*)
- Few-shot: give the model a handful of examples of the desired result
- Write individual instructions as a (numbered) list, one rule per line
- State the role when it helps ("You are a music journalist...")

Prompting Best Practices

You are a helpful assistant. What genre is this song? Answer briefly.



You are a music genre classifier.

Classify the song lyrics provided by the user into exactly one of the following genres: Country, Hip-Hop, Pop, R&B, Rock

Instructions:

1. Respond with exactly one label from the list above.
2. Use the exact spelling and capitalization shown.
3. Do not add any explanation, punctuation, or extra words.

Examples

<lyrics>

Driving home with a beer in my hand...

</lyrics>

Genre: Country

<lyrics>

Started from the bottom now we're here, started from the bottom now my whole team here...

</lyrics>

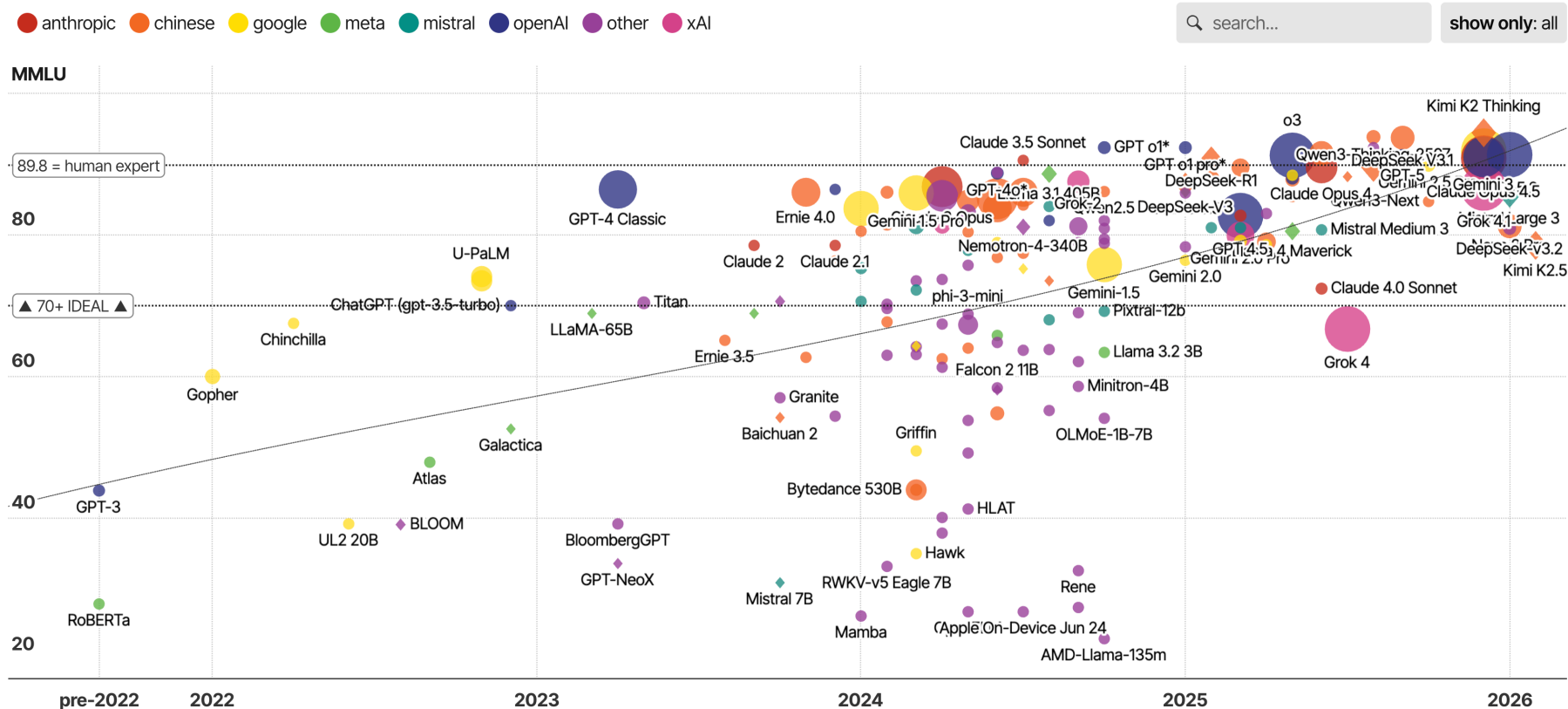
Genre: Hip-Hop



05.

LLM Landscape

LLM Landscape



Source: Information is beautiful

LLM Landscape

- Three big US players: OpenAI (GPT), Google (Gemini), Anthropic (Claude)
- Chinese open-weights models
 - DeepSeek, Qwen, Kimi, Minimax
- Some ~~open-source~~ open-weights models
 - LLaMA family (Meta), GPT OSS (OpenAI), Mistral, Gemma (Google), Phi (Microsoft)
- Open Source: training data, algorithms and weights are available
 - OLMo (AI2, Seattle), T5 (Google), Apertus (Swiss AI, ETH Zürich)

LLMs from Europe

- So far, the only competitive company and LLM is Mistral
 - Latest release: Mistral 3 and Mistral 3 Large (Dec 2025)
- Small endeavours from Germany and Switzerland
- There was/is one player from Heidelberg: Aleph Alpha

Braucht die deutsche Vorzeige-KI mehr Erziehung?

Die KI der deutschen Firma Aleph Alpha gilt als vielversprechendstes Produkt Europas. Doch sie generiert rassistische Texte. Das könnte zum Problem in Anwendungen werden.

- New server clusters are built right now (e.g. OpenEuroLLM including Jülich)
- Whether that's fruitful and maybe even necessary?

Statement on the US government directive to suspend access to Fable 5 and Mythos 5

12 Jun 2026

The US government, citing national security authorities, has issued an export control directive to suspend all access to Fable 5 and Mythos 5 by any foreign national, whether inside or outside the United States, including foreign national Anthropic employees. The net effect of this order is that we must abruptly disable Fable 5 and Mythos 5 for all our customers to ensure compliance. **Access to all other Anthropic models will not be affected.**

Using LLM Scenarios

Criterion	Direct Access Closed Source	Framework Contracts & Router Providers	Self-hosting
Cost	API fees per token; no infrastructure costs	Negotiated rates; min. volume may apply	Hardware, power, staff
Data Location	Data with provider (usually US / cloud)	Contractually regulated; EU servers possible	Full control; own data centre
Encryption	TLS in transit; provider manages keys	Contractually defined; often stronger than standard	Self-managed; full control
New Models	Available immediately on release	Depends on contract; possible delay	Manual update
Dependency	High: vendor lock-in, price & API changes	Medium; mitigated by contract	Open-source/weight dependency; community risk
Effort	Minimal; API integration only	Medium; contract management	Setup, operation, maintenance
Data Privacy & Compliance	GDPR critical; check DPA requirements	GDPR compliance possible; contractually secured	Full GDPR control; no third party
Scalability	Elastically scalable; no capacity limit	Contractually capped; easy to plan	Limited by hardware; expansion costly